FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WEDNESDAY 9 JULY 2025 – Laboratory Part

WITH SOLUTIONS

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- **The exam is open book: textbooks and personal notes are permitted.**
- **Pencil writing is permitted, except on the cover page (this sheet).**
- **Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.**
- **Time: lab part 1 h** – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory	syntax and semantics of languages, divided in four sections:	(75% of the final grade)
1)	regular expressions and finite automata	(1 exercise × 10 points)
2)	syntax analysis and parsing methodologies	(1 exercise × 10 points)
3)	grammar design – translation – semantic analysis	(2 exercises × 5 points)
4)	*bonus* question	(1 exercise × 5 points)
Lab	compiler design by Flex and Bison	(25% of the final grade)
1)	basic questions	(30 points)
2)	*bonus* question	(5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

   Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the ability to count the number of iterations performed in a while loop through a new **statement** called **`count_while`**.

   The syntax of this new statement is as shown in the example below:

```
int i, n;

i = 256;
n = 777;
n = count_while (i != 1) {
  i = i / 2;
  write(n); // prints 777, 8 times
}
write(n); // prints 8
```

   The `count_while` statement appears similar to a regular `while` statement but it is preceded by an assignment to a variable that — at the end of the execution of the loop — shall contain the number of iterations executed. In this example, the loop body executes 8 times, so the variable n has the value 8 at the end of the loop.

   Additionally, your implementation must behave as follows:

   - The `count_while` statement cannot appear inside a generic expression.
   - The destination variable, loop condition and loop body are arbitrary variable identifiers, expressions, and blocks respectively.
   - The parentheses around the expression are mandatory.
   - Before the equals sign, only variables can appear (not arrays or array elements).
   - The variable is only assigned once all the iterations of the loop are completed.

   Answer the following questions:

   (a) Define the tokens (and the related declarations in **scanner.l** and **parser.y**). (1 points)
   (b) Define the syntactic rules or the modifications required to the existing ones. (2 points)
   (c) Define the semantic actions needed to implement the required functionality. (22 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '−' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/parser.h b/acse/parser.h
--- a/acse/parser.h
+++ b/acse/parser.h
@@ -26,6 +26,12 @@ typedef struct {
   t_label *lExit; ///< Label to the first instruction after the loop.
 } t_whileStmt;

+typedef struct {
+  t_label *lLoop;
+  t_label *lExit;
```

```
+  t_regID rCounter;
+} t_countWhileStmt;
+
 /**
  * @}
  */
diff --git a/acse/parser.y b/acse/parser.y
--- a/acse/parser.y
+++ b/acse/parser.y
@@ -50,6 +50,7 @@ void yyerror(const char *msg)
   t_label *label;
   t_ifStmt ifStmt;
   t_whileStmt whileStmt;
+  t_countWhileStmt countWhileStmt;
 }

 /*
@@ -77,6 +78,7 @@ void yyerror(const char *msg)
 %token <label> DO
 %token <string> IDENTIFIER
 %token <integer> NUMBER
+%token <countWhileStmt> COUNT_WHILE

 /*
  * Non-terminal symbol semantic value type declarations
@@ -182,9 +184,46 @@ statement
   | return_statement SEMI
   | read_statement SEMI
   | write_statement SEMI
+  | count_while_statement
   | SEMI
 ;

+count_while_statement
+  : var_id ASSIGN COUNT_WHILE
+  {
+    // Check the type of the result variable. Note that even if we didn't do
+    // this, in the call to genStoreRegisterToVariable() the compiler would
+    // fail in a similar way, so we could omit this check with no consequences.
+    if (isArray($1)) {
+      yyerror("the destination of count_while must be a scalar");
+      YYERROR;
+    }
+    // Reserve a register initialized to zero which will count the iterations.
+    $3.rCounter = getNewRegister(program);
+    genLI(program, $3.rCounter, 0);
+    // Assign a label at the beginning of the loop for the back-edge.
+    $3.lLoop = createLabel(program);
+    assignLabel(program, $3.lLoop);
+  }
+  LPAR exp RPAR
+  {
+    // Generate a jump out of the loop if the condition is equal to zero.
+    $3.lExit = createLabel(program);
+    genBEQ(program, $6, REG_0, $3.lExit);
+  }
+  code_block
+  {
+    // Generate an increment of the counter register by 1.
+    genADDI(program, $3.rCounter, $3.rCounter, 1);
+    // Generate a jump back to the beginning of the loop after its body.
+    genJ(program, $3.lLoop);
+    // Assign the label to the end of the loop.
+    assignLabel(program, $3.lExit);
```

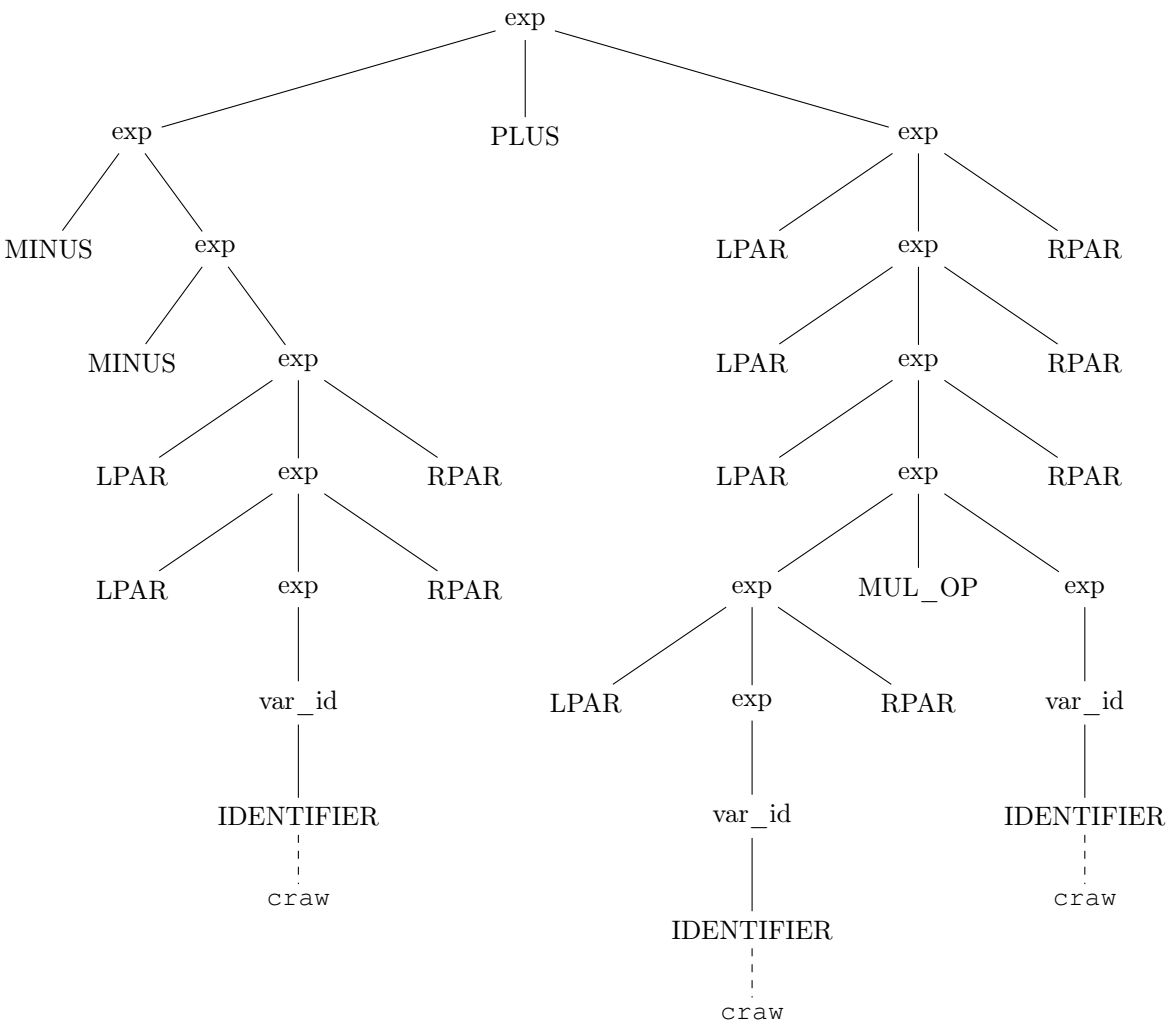
```
+    // Generate the assignment of the counter register to the variable.
+    genStoreRegisterToVariable(program, $1, $3.rCounter);
+  }
+;
+
 /* An assignment statement stores the value of an expression in the memory
  * location of a given scalar variable or array element. */
 assign_statement
diff --git a/acse/scanner.l b/acse/scanner.l
--- a/acse/scanner.l
+++ b/acse/scanner.l
@@ -81,6 +81,7 @@ ID                          [a-zA-Z_][a-zA-Z0-9_]*
 "return"                  { return RETURN; }
 "read"                    { return READ; }
 "write"                   { return WRITE; }
+"count_while"             { return COUNT_WHILE; }

 {ID}                      {
                             yylval.string = strdup(yytext);
diff --git a/tests/count_while/count_while.src b/tests/count_while/count_while.src
--- /dev/null
+++ b/tests/count_while/count_while.src
@@ -0,0 +1,9 @@
+int i, n;
+
+i = 256;
+n = 777;
+n = count_while (i != 1) {
+  i = i / 2;
+  write(n);
+}
+write(n); // prints 8
```

2. Given the following LANCE code snippet:

```
--((craw))+((((craw)*craw)))
```

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` *starting from the* `exp` *nonterminal.* (5 points)

Solution



2 Bonus Question (5 points)

1. Show how to modify your previous implementation of the `count_while` statement in order to make it a valid expression operator. Write in detail which lines of code are changed and how. Does the example code previously shown remain syntactically valid with this modification? (2 points)

2. Consider the following snippets of ACSE compiler code, which define a new **`st`** statement:

scanner.l

```
"st"                    { return ST; }
```

parser.y

```
%token ST

/* ... */

statement:
  /* ... */
  | st_statement SEMI
  | SEMI
;

st_statement
  : ST LPAR exp RPAR
  {
    if ($3 == 123) {
      genADDI(program, $3, $3, 1);
    } else {
      genSUBI(program, $3, $3, 1);
    }
  }
;
```

And the following LANCE program that uses the `st` statement defined as above:

st.src

```
int x;
x = 100;
st(x + 23);
st(123);
```

How many `ADDI` and how many `SUBI` instructions are generated when compiling the two `st` statements, and why? Only consider the code specific to the `st` statement, and not its expression argument. (3 points)

Solution

1. The grammatical rules must first of all be moved to the `exp` non-terminal:

   parser.y

   ```
   exp
     : COUNT_WHILE LPAR exp RPAR code_block
     | // ...
   ```

   The semantic actions remain the same (except for the semantic variables which are renumbered according to the new grammar). Exception is made for the last one where the call to `genStoreRegisterToVariable` is replaced with:

   parser.y

   ```
     {
       // ...
       $$ = $1.rCounter;
     }
   ```

2. The register ID in the semantic value of an `exp` non-terminal is never zero, and in fact is unrelated to the numeric value of the expression. The program is too short to reach register IDs that equal to 123, so in both cases the compiler generates a `SUBI` instruction. So the total is two `SUBI` instructions.

   The expression code generation generates an `ADD` instruction, not an `ADDI`, because all literal constants are first loaded in a register before being operated on (see the semantic actions for `NUMBER`). However, note that the text explicitly asked not to count the instructions generated by the `exp` nonterminal actions.